

DAY 2 - PYTHON COLLECTIONS

Interview Preparation Question Bank | Lists, Tuples, Sets, Dictionaries, Indexing & Slicing

Coverage	Day 2 only: Lists, Tuples, Sets, Dictionaries, Indexing, Slicing
Format	Question -> short interview-ready answer -> brief context
Priority	★ MUST ASK questions are grouped first for rapid revision
Target	149 total questions 46 ★ MUST ASK

How to use this question bank

First, revise the ★ MUST ASK section until you can answer aloud without notes. Then use the full bank to expose weak spots. For coding/trick questions, do not memorize the result - run the code yourself and explain why it happens.

★ MUST ASK - Rapid Revision Set

1. ★ MUST ASK: What are the main characteristics of a list?

Answer: Ordered, mutable, indexable, sliceable, and duplicates are allowed.

Why it matters: Know these characteristics without hesitation.

2. ★ MUST ASK: Why are lists called mutable?

Answer: Because their elements or structure can be changed after the list object is created.

Why it matters: Example: `items[0] = 100` changes the existing list.

3. ★ MUST ASK: How do you create a list?

Answer: Use square brackets: `[1, 2, 3]`. You can also use `list(iterable)`.

Why it matters: Common coding question.

4. ★ MUST ASK: What is list indexing?

Answer: Indexing accesses a single element by position. Python uses zero-based indexing.

Why it matters: First element is at index 0; negative indices count from the end.

5. ★ MUST ASK: What is list slicing?

Answer: Slicing extracts a portion of a sequence using `[start:stop:step]`, with stop excluded.

Why it matters: Very common in interviews.

6. ★ MUST ASK: `append()` vs `extend()`?

Answer: `append(x)` adds `x` as one element; `extend(iterable)` adds each element from the iterable.

Why it matters: Classic list interview question.

7. ★ MUST ASK: `append()` vs `insert()`?

Answer: `append(x)` adds at the end; `insert(i, x)` adds `x` at a specified position.

Why it matters: Know complexity at a high level: inserting near the front requires shifting elements.

8. ★ MUST ASK: `pop()` vs `remove()`?

Answer: `pop([i])` removes and returns by index (default: last); `remove(x)` deletes the first matching value.

Why it matters: One uses an index, the other a value.

9. ★ MUST ASK: `sort()` vs `sorted()`?

Answer: `sort()` modifies the list in place and returns `None`; `sorted()` returns a new sorted list and leaves the original unchanged.

Why it matters: Extremely common.

10. ★ MUST ASK: What is a shallow copy of a list?

Answer: A new outer list is created, but nested objects are still shared by reference.

Why it matters: Important when lists contain other mutable objects.

11. ★ MUST ASK: How do `copy()` and assignment differ for lists?

Answer: `b = a` aliases the same list; `b = a.copy()` creates a new outer list.

Why it matters: Test with `'is'` to see whether they are the same object.

12. ★ MUST ASK: What is the difference between a list copy and an alias?

Answer: An alias points to the same list; a copy creates a separate outer list.

Why it matters: Classic mutability question.

13. ★ MUST ASK: List comprehension vs traditional for loop?

Answer: They can express the same transformations; comprehensions are concise, while loops may be clearer for complex logic or side effects.

Why it matters: Interviewers often ask when readability matters.

14. ★ MUST ASK: List vs tuple?

Answer: Both are ordered and indexable; lists are mutable, tuples are immutable.

Why it matters: One of the most frequent Python interview questions.

15. ★ MUST ASK: Why would you choose a tuple over a list?

Answer: Use a tuple for fixed collections, to communicate immutability, or when an immutable/hashable sequence is useful.

Why it matters: Mention intent, not just performance claims.

16. ★ MUST ASK: What is extended unpacking?

Answer: The `*` target can collect multiple values during unpacking, e.g. `first, *middle, last = values`.

Why it matters: Common practical question.

17. ★ MUST ASK: What are the main characteristics of a set?

Answer: Elements are unique; the set is mutable; it is not a sequence, so it does not support positional indexing; set operations are supported.

Why it matters: Know the trade-off: uniqueness and fast membership over positional access.

18. ★ MUST ASK: Why are sets useful?

Answer: They are useful for removing duplicates, membership tests, and operations such as union and intersection.

Why it matters: Very relevant to data work.

19. ★ MUST ASK: Why must set elements be hashable?

Answer: Sets use hashing to place and locate elements efficiently, so their elements must have a stable hash value and equality behavior.

Why it matters: Connects directly to dictionaries.

20. ★ MUST ASK: What is the difference between `{}` and `set()`?

Answer: `{}` is an empty dictionary; `set()` creates an empty set.

Why it matters: Must memorize.

21. ★ MUST ASK: `remove()` vs `discard()`?

Answer: `remove` raises `KeyError` when absent; `discard` does not.

Why it matters: Frequently tested behavior.

22. ★ MUST ASK: How do sets compare with lists for membership testing?

Answer: Set membership is typically $O(1)$ average-case, while list membership is $O(n)$.

Why it matters: Important for performance and data-structure choice.

23. ★ MUST ASK: What are the main characteristics of a dictionary?

Answer: It stores key-value pairs, keys are unique, it is mutable, and modern Python preserves insertion order for iteration.

Why it matters: High-frequency interview topic.

24. ★ MUST ASK: Can dictionary keys be duplicated?

Answer: No. Assigning a value to an existing key replaces the previous value.

Why it matters: Very common trick question.

25. ★ MUST ASK: Why must dictionary keys be hashable?

Answer: Dictionaries use hashing to locate keys efficiently, so keys need stable hash/equality behavior.

Why it matters: Core internal concept.

26. ★ MUST ASK: `get()` vs `[]` for dictionaries?

Answer: `d[key]` raises `KeyError` if missing; `d.get(key, default)` returns a default instead.

Why it matters: Very common and practical.

27. ★ MUST ASK: How do you iterate over a dictionary?

Answer: Use `for key in d` for keys, `for value in d.values()` for values, or `for key, value in d.items()` for pairs.

Why it matters: Interviewers often test practical fluency.

28. ★ MUST ASK: How do you merge dictionaries?

Answer: Common approaches include `d1.update(d2)`, `{**d1, **d2}`, or the `|` operator in modern Python.

Why it matters: Know that later values win on key conflicts.

29. ★ MUST ASK: `pop()` vs `popitem()` for dictionaries?

Answer: `pop(key)` removes a specified key and returns its value; `popitem()` removes and returns the last inserted key-value pair in modern Python.

Why it matters: Common interview comparison.

30. ★ MUST ASK: What is the typical average time complexity of dictionary lookup?

Answer: $O(1)$ average-case, assuming good hashing behavior; pathological collision behavior can degrade performance.

Why it matters: Useful for data-structure selection.

31. ★ MUST ASK: What is zero-based indexing?

Answer: The first element is at index 0, the second at 1, and so on.

Why it matters: Fundamental Python sequence behavior.

32. ★ MUST ASK: What is negative indexing?

Answer: Negative indices count from the end: -1 is the last element, -2 the second-last, etc.

Why it matters: Frequently used in coding rounds.

33. ★ MUST ASK: What is slicing?

Answer: Slicing extracts a range from a sequence using `start:stop:step`.

Why it matters: The stop bound is excluded.

34. ★ MUST ASK: Why is the stop index excluded in slicing?

Answer: It makes ranges composable and aligns with Python's half-open interval convention.

Why it matters: Good conceptual explanation.

35. ★ MUST ASK: What does `a[::-1]` do?

Answer: It returns a reversed sequence using a step of -1.

Why it matters: Common coding question.

36. ★ MUST ASK: What is the difference between indexing and slicing?

Answer: Indexing returns one element; slicing returns a subsequence.

Why it matters: Very common.

37. ★ MUST ASK: List vs tuple vs set vs dictionary?

Answer: List: ordered, mutable sequence with duplicates. Tuple: ordered, immutable sequence with duplicates. Set: unique elements, optimized for membership/set operations. Dictionary: key-value mapping with unique keys.

Why it matters: Know this comparison as a table in your head.

38. ★ MUST ASK: Which collection would you choose for unique customer IDs?

Answer: A set when you primarily need uniqueness and membership testing.

Why it matters: Data-science-oriented design question.

39. ★ MUST ASK: Which collection would you choose for a fixed coordinate pair?

Answer: A tuple is a natural choice because the structure is fixed and ordered.

Why it matters: Example: `(25.3, 55.4)`.

40. ★ MUST ASK: Which collection would you choose for fast key-based lookup?

Answer: A dictionary.

Why it matters: Example: `customer_id -> customer_record`.

41. ★ MUST ASK: Which collection would you choose when order and duplicates matter and you need updates?

Answer: A list.

Why it matters: Common workflow choice.

42. ★ MUST ASK: What happens here? `a = [1,2]`; `b = a`; `b.append(3)`.

Answer: Both `a` and `b` now reference the same list, so both show `[1,2,3]`.

Why it matters: Tests aliasing and mutability.

43. ★ MUST ASK: What happens here? `a = [1,2]`; `b = a.copy()`; `b.append(3)`.

Answer: `a` remains `[1,2]`, while `b` becomes `[1,2,3]`, because the outer list was copied.

Why it matters: Tests shallow copy.

44. ★ MUST ASK: What happens here? `x = [[1,2],[3,4]]`; `y = x.copy()`; `y[0].append(99)`.

Answer: The nested list is shared, so `x[0]` also becomes `[1,2,99]`.

Why it matters: Classic shallow-copy trap.

45. ★ MUST ASK: Why does `{}` create a dictionary instead of a set?

Answer: The literal `{}` is defined as an empty dictionary; an empty set requires `set()`.

Why it matters: Classic Python syntax trap.

46. ★ MUST ASK: What is the difference between modifying a mutable object and rebinding a variable?

Answer: Mutation changes the existing object; rebinding makes the variable refer to a different object.

Why it matters: Core mental model for Python collections.

Full Question Bank

Lists

1. What is a Python list?

Answer: A list is an ordered, mutable sequence that can hold heterogeneous objects and duplicate values.

Context: Core definition.

2. ★ MUST ASK: What are the main characteristics of a list?

Answer: Ordered, mutable, indexable, sliceable, and duplicates are allowed.

Context: Know these characteristics without hesitation.

3. ★ MUST ASK: Why are lists called mutable?

Answer: Because their elements or structure can be changed after the list object is created.

Context: Example: `items[0] = 100` changes the existing list.

4. ★ MUST ASK: How do you create a list?

Answer: Use square brackets: `[1, 2, 3]`. You can also use `list(iterable)`.

Context: Common coding question.

5. Can a list contain different data types?

Answer: Yes. A list can contain integers, strings, other lists, objects, and mixed types.

Context: Example: `[1, 'ML', 3.5]`.

6. Can a list contain another list?

Answer: Yes. Lists can be nested.

Context: Example: `[[1,2], [3,4]]`.

7. ★ MUST ASK: What is list indexing?

Answer: Indexing accesses a single element by position. Python uses zero-based indexing.

Context: First element is at index 0; negative indices count from the end.

8. ★ MUST ASK: What is list slicing?

Answer: Slicing extracts a portion of a sequence using `[start:stop:step]`, with stop excluded.

Context: Very common in interviews.

9. What does `my_list[-1]` return?

Answer: The last element of the list.

Context: Negative indices count from the right.

10. What happens when you assign to `list[0]`?

Answer: The first element is replaced because lists are mutable.

Context: Example: `a[0] = 99`.

11. ★ MUST ASK: `append()` vs `extend()`?

Answer: `append(x)` adds `x` as one element; `extend(iterable)` adds each element from the iterable.

Context: Classic list interview question.

12. ★ MUST ASK: `append()` vs `insert()`?

Answer: `append(x)` adds at the end; `insert(i, x)` adds `x` at a specified position.

Context: Know complexity at a high level: inserting near the front requires shifting elements.

13. What does `remove(x)` do?

Answer: It removes the first matching value and raises `ValueError` if the value is absent.

Context: Distinguish from `pop()`.

14. ★ MUST ASK: `pop()` vs `remove()`?

Answer: `pop([i])` removes and returns by index (default: last); `remove(x)` deletes the first matching value.

Context: One uses an index, the other a value.

15. What does `del` do with a list?

Answer: It deletes an item, slice, or even the whole list binding.

Context: Example: `del a[1]`.

16. What does `clear()` do?

Answer: It removes all elements from the list, leaving an empty list object.

Context: Different from deleting the variable name.

17. ★ MUST ASK: `sort()` vs `sorted()`?

Answer: `sort()` modifies the list in place and returns `None`; `sorted()` returns a new sorted list and leaves the original unchanged.

Context: Extremely common.

18. Does `list.sort()` return the sorted list?

Answer: No. It returns `None` because it sorts in place.

Context: Frequent trick question.

19. How do you sort a list in descending order?

Answer: Use `list.sort(reverse=True)` or `sorted(iterable, reverse=True)`.

Context: Know both in-place and non-mutating forms.

20. ★ MUST ASK: What is a shallow copy of a list?

Answer: A new outer list is created, but nested objects are still shared by reference.

Context: Important when lists contain other mutable objects.

21. ★ MUST ASK: How do `copy()` and assignment differ for lists?

Answer: `b = a` aliases the same list; `b = a.copy()` creates a new outer list.

Context: Test with 'is' to see whether they are the same object.

22. What is a deep copy?

Answer: `copy.deepcopy` creates a recursively independent copy of nested objects, subject to object-specific behavior.

Context: Know when shallow copying is insufficient.

23. ★ MUST ASK: What is the difference between a list copy and an alias?

Answer: An alias points to the same list; a copy creates a separate outer list.

Context: Classic mutability question.

24. How do you remove duplicates from a list?

Answer: A common approach is `list(set(values))`, but that does not preserve original order. For order-preserving deduplication, use a loop/dict-based approach such as `list(dict.fromkeys(values))`.

Context: Good practical data-science question.

25. How do you find the length of a list?

Answer: Use `len(list_obj)`.

Context: $O(1)$ for built-in Python lists.

26. How do you check whether an item exists in a list?

Answer: Use the `in` operator.

Context: Example: `x in items`.

27. What happens if you access an invalid list index?

Answer: Python raises `IndexError`.

Context: Common runtime-error question.

28. Can list indices be negative?

Answer: Yes. `-1` is the last element, `-2` the second-last, and so on.

Context: Useful for reverse access.

29. What is list comprehension?

Answer: A compact syntax for constructing a list from an iterable, optionally with a condition.

Context: Example: `[x*x for x in nums]`.

30. ★ MUST ASK: List comprehension vs traditional for loop?

Answer: They can express the same transformations; comprehensions are concise, while loops may be clearer for complex logic or side effects.

Context: Interviewers often ask when readability matters.

31. Can a list be used as a dictionary key?

Answer: No. Lists are mutable and therefore unhashable.

Context: This connects lists to hashing and dictionaries.

32. Can a list contain None?

Answer: Yes. None is just another object that can be stored in a list.

Context: Example: [1, None, 3].

33. What is the typical time complexity of membership testing in a list?

Answer: $O(n)$ average/worst-case because Python may scan elements sequentially.

Context: Useful for choosing data structures.

34. What is the typical time complexity of append()?

Answer: Amortized $O(1)$.

Context: Individual resizes can cost more, but amortized performance is constant.

35. Why can inserting at the beginning of a list be slower than appending?

Answer: Elements after the insertion point may need to be shifted, making front insertion $O(n)$.

Context: Important performance intuition.

Tuples

36. What is a Python tuple?

Answer: A tuple is an ordered, immutable sequence that can contain duplicates and mixed types.

Context: Core definition.

37. ★ MUST ASK: List vs tuple?

Answer: Both are ordered and indexable; lists are mutable, tuples are immutable.

Context: One of the most frequent Python interview questions.

38. ★ MUST ASK: Why would you choose a tuple over a list?

Answer: Use a tuple for fixed collections, to communicate immutability, or when an immutable/hashable sequence is useful.

Context: Mention intent, not just performance claims.

39. Can a tuple contain mutable objects?

Answer: Yes. The tuple structure is immutable, but an object inside it can still be mutable.

Context: Example: ([1,2], 3).

40. What is tuple unpacking?

Answer: Assigning tuple elements to multiple variables in one statement.

Context: Example: a, b = (10, 20).

41. ★ MUST ASK: What is extended unpacking?

Answer: The * target can collect multiple values during unpacking, e.g. first, *middle, last = values.

Context: Common practical question.

42. How do you create a single-element tuple?

Answer: Use a trailing comma: (5,). Parentheses alone do not make the tuple.

Context: Classic trick question.

43. What is the difference between (5) and (5,)?

Answer: (5) is just the integer 5; (5,) is a one-element tuple.

Context: Must know.

44. Can tuples be sliced and indexed?

Answer: Yes. Tuples support sequence indexing and slicing.

Context: They behave like other sequences in this respect.

45. Can tuples be dictionary keys?

Answer: Yes, if the tuple is hashable, meaning its elements are hashable.

Context: A tuple containing a list is not hashable.

46. Why can tuples be used as dictionary keys but lists cannot?

Answer: Dictionary keys must be hashable; tuples can be hashable when their elements are hashable, while lists are mutable and unhashable.

Context: High-value conceptual question.

47. How do you convert a list to a tuple?

Answer: Use tuple(list_obj).

Context: Creates a tuple containing the list's elements.

48. How do you convert a tuple to a list?

Answer: Use `list(tuple_obj)`.

Context: Creates a new list.

49. Are tuples more memory-efficient than lists?

Answer: Typically yes for comparable fixed contents because tuples have a smaller, simpler structure, but exact memory use depends on contents and implementation.

Context: Do not claim a universal dramatic speed advantage.

50. Are tuples faster than lists?

Answer: They can be slightly faster for some operations because they are immutable and simpler internally, but the practical difference depends on the workload.

Context: Interview-safe answer.

Sets

51. What is a Python set?

Answer: A set is a mutable collection of unique elements designed for efficient membership testing and set operations.

Context: Core definition.

52. ★ MUST ASK: What are the main characteristics of a set?

Answer: Elements are unique; the set is mutable; it is not a sequence, so it does not support positional indexing; set operations are supported.

Context: Know the trade-off: uniqueness and fast membership over positional access.

53. ★ MUST ASK: Why are sets useful?

Answer: They are useful for removing duplicates, membership tests, and operations such as union and intersection.

Context: Very relevant to data work.

54. Does a set allow duplicate elements?

Answer: No. Adding an element already present has no effect on set membership.

Context: Uniqueness is fundamental.

55. Can a set contain a list?

Answer: No. A list is unhashable, so it cannot be a set element.

Context: Set elements must be hashable.

56. Can a set contain a tuple?

Answer: Yes, provided all elements inside that tuple are hashable.

Context: Example: `{(1,2), (3,4)}`.

57. ★ MUST ASK: Why must set elements be hashable?

Answer: Sets use hashing to place and locate elements efficiently, so their elements must have a stable hash value and equality behavior.

Context: Connects directly to dictionaries.

58. How do you create an empty set?

Answer: Use `set()`. `{}` creates an empty dictionary, not an empty set.

Context: Classic Python interview trap.

59. ★ MUST ASK: What is the difference between `{}` and `set()`?

Answer: `{}` is an empty dictionary; `set()` creates an empty set.

Context: Must memorize.

60. How do you remove an element from a set?

Answer: `remove(x)` deletes `x` but raises `KeyError` if `x` is missing; `discard(x)` does nothing if `x` is missing.

Context: Common question.

61. ★ MUST ASK: `remove()` vs `discard()`?

Answer: `remove` raises `KeyError` when absent; `discard` does not.

Context: Frequently tested behavior.

62. What does `pop()` do on a set?

Answer: It removes and returns an arbitrary element. You should not rely on a predictable order.

Context: Important difference from `list.pop()`.

63. What is union?

Answer: The set containing all elements from both sets, excluding duplicates.

Context: Use `|` or `union()`.

64. What is intersection?

Answer: The set of elements common to both sets.

Context: Use `&` or `intersection()`.

65. What is difference?

Answer: Elements in the left set that are not in the right set.

Context: Use `-` or `difference()`.

66. What is symmetric difference?

Answer: Elements in either set but not in both.

Context: Use `^` or `symmetric_difference()`.

67. ★ MUST ASK: How do sets compare with lists for membership testing?

Answer: Set membership is typically $O(1)$ average-case, while list membership is $O(n)$.

Context: Important for performance and data-structure choice.

68. Can you index a set?

Answer: No. Sets do not support sequence-style positional indexing.

Context: Trying `my_set[0]` raises `TypeError`.

69. Are sets ordered?

Answer: Do not rely on a set having a meaningful positional order for indexing or business logic.

Context: Interview-safe framing.

70. How do you convert a list to a set?

Answer: Use `set(list_obj)`.

Context: Useful for deduplication.

71. What caveat exists when converting a list to a set?

Answer: Duplicates are removed, and the original positional ordering should not be relied upon.

Context: Important in data-cleaning discussions.

72. Can a set be a dictionary key?

Answer: A regular set cannot because it is mutable and unhashable; a `frozenset` can be a key if its elements are hashable.

Context: Useful distinction.

Dictionaries

73. What is a Python dictionary?

Answer: A dictionary is a mutable mapping from keys to values.

Context: Core definition.

74. ★ MUST ASK: What are the main characteristics of a dictionary?

Answer: It stores key-value pairs, keys are unique, it is mutable, and modern Python preserves insertion order for iteration.

Context: High-frequency interview topic.

75. ★ MUST ASK: Can dictionary keys be duplicated?

Answer: No. Assigning a value to an existing key replaces the previous value.

Context: Very common trick question.

76. Can dictionary values be duplicated?

Answer: Yes. Multiple keys can map to equal values.

Context: Common contrast with keys.

77. What types can be dictionary keys?

Answer: Hashable objects can be keys, such as strings, numbers, tuples of hashable elements, and `frozensets`.

Context: Lists, sets, and dictionaries cannot be keys.

78. ★ MUST ASK: Why must dictionary keys be hashable?

Answer: Dictionaries use hashing to locate keys efficiently, so keys need stable hash/equality behavior.

Context: Core internal concept.

79. Can a list be a dictionary key?

Answer: No, because lists are mutable and unhashable.

Context: Classic question.

80. Can a tuple be a dictionary key?

Answer: Yes, if all of its elements are hashable.

Context: Tuple-with-list exception matters.

81. How do you create a dictionary?

Answer: Use {key: value} pairs, dict(), or other mapping constructors.

Context: Example: {'name': 'Asha', 'age': 25}.

82. How do you access a dictionary value?

Answer: Use d[key] when the key is expected to exist; missing keys raise KeyError.

Context: Know the safer alternative: get().

83. ★ MUST ASK: get() vs [] for dictionaries?

Answer: d[key] raises KeyError if missing; d.get(key, default) returns a default instead.

Context: Very common and practical.

84. What does setdefault() do?

Answer: It returns the value for a key if present; otherwise it inserts the key with the supplied default and returns that default.

Context: Useful but less frequently used.

85. What does keys() return?

Answer: A dynamic view of dictionary keys.

Context: Views reflect changes to the dictionary.

86. What does values() return?

Answer: A dynamic view of dictionary values.

Context: Not a plain list by default.

87. What does items() return?

Answer: A dynamic view of key-value pairs as tuples.

Context: Common in loops: for k, v in d.items().

88. ★ MUST ASK: How do you iterate over a dictionary?

Answer: Use for key in d for keys, for value in d.values() for values, or for key, value in d.items() for pairs.

Context: Interviewers often test practical fluency.

89. What happens when you assign to an existing dictionary key?

Answer: Its old value is replaced with the new value.

Context: Example: d['x'] = 10 then d['x'] = 20.

90. What does update() do?

Answer: It adds or replaces key-value pairs from another mapping or iterable of pairs.

Context: Can merge data into a dictionary.

91. ★ MUST ASK: How do you merge dictionaries?

Answer: Common approaches include d1.update(d2), {**d1, **d2}, or the | operator in modern Python.

Context: Know that later values win on key conflicts.

92. What does the | operator do for dictionaries?

Answer: It creates a new dictionary by merging mappings, with values from the right operand overriding duplicate keys.

Context: Python 3.9+ feature.

93. What does |= do for dictionaries?

Answer: It updates the left dictionary in place with key-value pairs from the right operand.

Context: Know distinction from |.

94. How do you remove a dictionary entry?

Answer: Use del d[key], pop(key), or popitem().

Context: Know return/exception differences.

95. ★ MUST ASK: pop() vs popitem() for dictionaries?

Answer: pop(key) removes a specified key and returns its value; popitem() removes and returns the last inserted key-value pair in modern Python.

Context: Common interview comparison.

96. What happens if you del a missing dictionary key?

Answer: Python raises KeyError.

Context: Common error behavior.

97. How do you check if a key exists?

Answer: Use key in d.

Context: Membership tests dictionary keys by default.

98. Does 'x' in a dictionary check keys or values?

Answer: Keys. To test values, use x in d.values().

Context: Very common practical question.

99. ★ MUST ASK: What is the typical average time complexity of dictionary lookup?

Answer: O(1) average-case, assuming good hashing behavior; pathological collision behavior can degrade performance.

Context: Useful for data-structure selection.

100. Why is dictionary lookup usually fast?

Answer: Because the dictionary uses a hash-table-based design to locate keys efficiently.

Context: Conceptual internal model.

101. What happens when two distinct keys have the same hash?

Answer: The dictionary handles the collision using its internal collision-resolution strategy and still compares keys for equality.

Context: You only need the concept, not implementation details.

102. Why are dictionaries useful in data science?

Answer: They naturally represent structured records, configuration, feature maps, counters, lookups, and JSON-like data.

Context: Practical connection.

Indexing & Slicing

103. ★ MUST ASK: What is zero-based indexing?

Answer: The first element is at index 0, the second at 1, and so on.

Context: Fundamental Python sequence behavior.

104. ★ MUST ASK: What is negative indexing?

Answer: Negative indices count from the end: -1 is the last element, -2 the second-last, etc.

Context: Frequently used in coding rounds.

105. Which built-in collection types support positional indexing?

Answer: Lists and tuples do. Sets do not. Dictionaries are mappings, not positional sequences, although iteration preserves insertion order.

Context: Important conceptual distinction.

106. ★ MUST ASK: What is slicing?

Answer: Slicing extracts a range from a sequence using start:stop:step.

Context: The stop bound is excluded.

107. ★ MUST ASK: Why is the stop index excluded in slicing?

Answer: It makes ranges composable and aligns with Python's half-open interval convention.

Context: Good conceptual explanation.

108. What does a[2:5] mean?

Answer: Elements at indices 2, 3, and 4.

Context: Stop index 5 is excluded.

109. What does a[:3] mean?

Answer: From the beginning through index 2.

Context: Start defaults to the beginning.

110. What does a[3:] mean?

Answer: From index 3 to the end.

Context: Stop defaults to the end.

111. What does `a[:]` mean?

Answer: A full slice; for a list, it creates a shallow copy of the list.

Context: Important copy behavior.

112. ★ MUST ASK: What does `a[::-1]` do?

Answer: It returns a reversed sequence using a step of -1.

Context: Common coding question.

113. What happens if the step in a slice is zero?

Answer: Python raises `ValueError` because a slice step cannot be zero.

Context: Good edge-case question.

114. What happens if slice bounds go beyond the sequence length?

Answer: Slicing gracefully clamps to valid boundaries instead of raising `IndexError`.

Context: Important contrast with direct indexing.

115. What happens when an index is out of range?

Answer: Direct indexing raises `IndexError`.

Context: Contrast with slicing.

116. Can strings be indexed and sliced?

Answer: Yes. Strings are sequences and support indexing/slicing, but strings are immutable.

Context: Connects Day 1 and Day 2.

117. Can tuples be indexed and sliced?

Answer: Yes. They are ordered immutable sequences.

Context: Common comparison question.

118. Can sets be sliced?

Answer: No. Sets do not support sequence slicing.

Context: Because sets have no positional indexing interface.

119. ★ MUST ASK: What is the difference between indexing and slicing?

Answer: Indexing returns one element; slicing returns a subsequence.

Context: Very common.

120. What does step control in a slice?

Answer: The interval between selected positions and the direction when negative.

Context: Example: `a[::2]` takes every second element.

121. What does `a[1:8:2]` select?

Answer: Indices 1, 3, 5, and 7 if they exist.

Context: Good test of stop/step rules.

122. What does `a[-5:-1]` mean?

Answer: It selects elements from the fifth-last position up to but not including the last element.

Context: Negative slicing is a common coding check.

123. What is the difference between `a[:]` and `list(a)`?

Answer: For a list, both create a shallow copy of the outer list; `list(a)` also works from any iterable that can be converted into a list.

Context: Use case depends on the input type.

124. Does slicing a list perform a deep copy?

Answer: No. It creates a shallow copy of the outer list.

Context: Nested mutable elements are still shared.

Cross-collection comparisons & practical choices

125. ★ MUST ASK: List vs tuple vs set vs dictionary?

Answer: List: ordered, mutable sequence with duplicates. Tuple: ordered, immutable sequence with duplicates. Set: unique elements, optimized for membership/set operations. Dictionary: key-value mapping with unique keys.

Context: Know this comparison as a table in your head.

126. ★ MUST ASK: Which collection would you choose for unique customer IDs?

Answer: A set when you primarily need uniqueness and membership testing.

Context: Data-science-oriented design question.

127. ★ MUST ASK: Which collection would you choose for a fixed coordinate pair?

Answer: A tuple is a natural choice because the structure is fixed and ordered.

Context: Example: (25.3, 55.4).

128. ★ MUST ASK: Which collection would you choose for fast key-based lookup?

Answer: A dictionary.

Context: Example: `customer_id -> customer_record`.

129. ★ MUST ASK: Which collection would you choose when order and duplicates matter and you need updates?

Answer: A list.

Context: Common workflow choice.

130. Why might a set be preferable to `list(set(...))` only after understanding the data semantics?

Answer: Converting to a set destroys duplicate information and should only be done when uniqueness is actually desired.

Context: Good data-cleaning reasoning.

131. When is dictionary membership useful?

Answer: When checking whether a key exists in a mapping, often for lookup tables, counters, and configuration.

Context: Use `key in d`.

132. When is list membership acceptable despite $O(n)$ search?

Answer: When lists are small, order matters, or a sequential scan is appropriate and simple.

Context: Don't blindly optimize everything.

133. Why can dictionary/set membership be much faster than list membership?

Answer: They use hashing-based lookup on average, while lists scan elements sequentially.

Context: Performance intuition.

134. How do you count frequencies efficiently using a dictionary?

Answer: Use a mapping from item to count, e.g. `counts[x] = counts.get(x, 0) + 1`.

Context: Very common practical question.

135. How do you preserve order while removing duplicates?

Answer: A common approach is `list(dict.fromkeys(values))` when elements are suitable dictionary keys.

Context: Useful practical pattern.

136. What is nested data in Python collections?

Answer: Collections can contain other collections, such as list of dictionaries or dictionary of lists.

Context: Very common in JSON/data-science work.

137. How would you represent a table-like record in pure Python?

Answer: A list of dictionaries is a common lightweight representation: each dictionary represents a row with named fields.

Context: Useful before Pandas.

138. How would you represent a one-to-many relationship?

Answer: A dictionary can map one key to a list/set of related values.

Context: Example: `customer_id -> order_ids`.

Scenario / tricky questions

139. ★ MUST ASK: What happens here? `a = [1,2]; b = a; b.append(3)`.

Answer: Both `a` and `b` now reference the same list, so both show `[1,2,3]`.

Context: Tests aliasing and mutability.

140. ★ MUST ASK: What happens here? `a = [1,2]; b = a.copy(); b.append(3)`.

Answer: `a` remains `[1,2]`, while `b` becomes `[1,2,3]`, because the outer list was copied.

Context: Tests shallow copy.

141. ★ MUST ASK: What happens here? `x = [[1,2],[3,4]]; y = x.copy(); y[0].append(99)`.

Answer: The nested list is shared, so `x[0]` also becomes `[1,2,99]`.

Context: Classic shallow-copy trap.

142. What happens when you assign a new value to d['name'] in a dictionary?

Answer: The previous value for that key is replaced.

Context: Keys are unique.

143. What happens when you add an existing value to a set?

Answer: The set remains unchanged in terms of membership; duplicates are not stored.

Context: Uniqueness property.

144. What does d.get('missing') return by default?

Answer: None, unless a different default is provided.

Context: Unlike d['missing'], it does not raise KeyError.

145. ★ MUST ASK: Why does {} create a dictionary instead of a set?

Answer: The literal {} is defined as an empty dictionary; an empty set requires set().

Context: Classic Python syntax trap.

146. Can dictionary values be lists?

Answer: Yes. Dictionary values can be any Python object.

Context: Only keys are restricted to hashable objects.

147. Can dictionary keys be bool and int with the same numeric value?

Answer: True and 1 compare equal and have the same hash, so they refer to the same dictionary key slot.

Context: Advanced but practical Python behavior; know the principle, not trivia.

148. Why is list sorting in place relevant in interviews?

Answer: Because in-place methods often return None, so assigning x = x.sort() can accidentally make x become None.

Context: Common bug pattern.

149. ★ MUST ASK: What is the difference between modifying a mutable object and rebinding a variable?

Answer: Mutation changes the existing object; rebinding makes the variable refer to a different object.

Context: Core mental model for Python collections.

Day 2 Final Interview Checklist

1. ■ Can I define list, tuple, set, and dictionary in one sentence each?
2. ■ Can I clearly explain mutable vs immutable using lists and tuples?
3. ■ Can I explain append vs extend, sort vs sorted, pop vs remove?
4. ■ Can I explain why set/dictionary elements or keys must be hashable?
5. ■ Can I explain == vs is using mutable collection examples?
6. ■ Can I explain aliasing, shallow copy, and nested mutable objects?
7. ■ Can I explain dictionary lookup and set membership at a high level?
8. ■ Can I write and explain slices such as [2:5], [:3], [3:], [::2], and [::-1]?
9. ■ Can I explain why direct indexing can raise IndexError while slicing usually does not?
10. ■ Can I choose the right collection for a practical data-science scenario and justify the choice?

Day 2 scope note

This question bank is intentionally restricted to the Day 2 topics in the 120-day plan: Lists, Tuples, Sets, Dictionaries, Indexing and Slicing. Control flow and functions belong to Day 3 in the plan and are excluded here.